UNIVERSIDAD NACIONAL DE COSTA RICA

ESCUELA DE INFORMÁTICA

Proyecto Programado N.º 2

Sistema experto inteligente de diagnóstico automotriz

Alternativa A · Programación lógica en Prolog y programación estructurada en C

Paradigmas de Programación · II Ciclo 2026

Manual técnico

Problema, arquitectura, conocimiento, reglas, integración y decisiones de diseño

Integrantes

Danny Alfaro Sánchez

Caleb Alfaro Fulopp

Judith Zamora Hidalgo

II Ciclo, 2026

Contenido

1. Descripción del problema 3
1.1 Ejemplo del enunciado 3
1.2 Alcance 3
2. Arquitectura 4
2.1 Modelo de procesos 4
2.2 Módulos del back-end (Prolog) 4
2.3 Módulos del front-end y la integración (C) 5
3. Base de conocimiento 5
3.1 Hechos dinámicos de la sesión 6
3.2 Validación de la KB 6
4. Motor de inferencia y reglas 7
4.1 Capa 1: de síntomas a componentes (Naive Bayes) 7
4.2 Capa 2: pruebas adicionales (noisy-OR) 7
4.3 Capa 3: acciones correctivas (noisy-OR) 7
4.4 Ciclo de razonamiento R1–R11 7
4.5 Retrocesos 8
4.6 Reglas compuestas 8
5. Recursividad 8
5.1 Problema 1: el ciclo de razonamiento R1–R11 8
5.2 Problema 2: combinación de evidencias sobre listas 9
5.3 Otras reglas recursivas 9
6. Listas y backtracking 9
6.1 Operaciones con listas 9
6.2 Escenarios de backtracking 10
7. Consultas lógicas 10
8. Explicación del razonamiento 11
9. Integración C–Prolog 11
9.1 Contrato JSON 11
9.2 Operaciones 11
9.3 Manejo de errores (RF-13) 12
10. Front-end en C 12
10.1 Interfaz de consola 13
10.2 Ventana gráfica 13
11. Persistencia y reportes 14
12. Pruebas 14
13. Cumplimiento de requisitos 14

13.1 Requisitos mínimos del motor lógico (sección 5) 14
13.2 Requisitos funcionales 15
13.3 Restricciones técnicas (sección 7) 15
14. Decisiones de diseño 16
15. Estructura del repositorio 17

1. Descripción del problema

El sistema corresponde a la **Alternativa A** del enunciado: un sistema experto que identifica posibles fallas mecánicas, eléctricas y electrónicas de un vehículo a partir de los síntomas que observa el usuario y de los resultados de pruebas adicionales. Las salidas son **hipótesis diagnósticas, no certezas**: cada conclusión se presenta con su probabilidad, su justificación y la recomendación de confirmarla con un técnico.

El usuario registra las características del vehículo, marca los síntomas observados (presente, ausente o desconocido) y pide un diagnóstico. El motor de Prolog calcula la probabilidad de cada componente, descarta las hipótesis incompatibles, recomienda la prueba más útil para el componente en foco y, cuando el usuario registra el resultado, vuelve a razonar. Al final presenta las acciones correctivas, explica el razonamiento paso a paso, guarda el diagnóstico en un historial y genera un reporte textual.

1.1 Ejemplo del enunciado

Entrada del enunciado: el vehículo no arranca; las luces funcionan; el motor de arranque no gira; el voltaje de batería es adecuado. Con los síntomas no_arranca = presente, motor_no_gira = presente y faros_tenues = ausente, el sistema responde:

Componente	Probabilidad	Estado
Relé de arranque	85,4 %	confirmado
Circuito de control del arranque (interruptor de encendido y cableado)	85,4 %	confirmado
Solenoide del motor de arranque	33,3 %	incierto (hipótesis)
Motor de arranque	33,3 %	incierto

Prueba recomendada: "Probar el solenoide del motor de arranque" (relevancia 70 %). Es la misma conclusión del ejemplo de salida del enunciado (relé, solenoide o circuito de control), pero deducida por las reglas a partir de la base de conocimiento, no almacenada como respuesta.

1.2 Alcance

- Base de conocimiento de **80 componentes**, **50 síntomas**, **152 pruebas** con su hallazgo y **111 acciones correctivas**.
- Razonamiento probabilístico (Naive Bayes binario con suavizado de Laplace) combinado con un ciclo de inferencia recursivo R1–R11 con tres tipos de retroceso (RB1, RB2, RB3).

- Interfaz en C de **consola** (menú de texto en la terminal) y una ventana gráfica con 8 pantallas, ambas sobre el mismo controlador; validación de todas las entradas antes de consultar a Prolog y manejo de errores sin cerrar la aplicación.
- Persistencia del historial en datos/registros.txt y reportes en datos/reportes/.

2. Arquitectura

La arquitectura sigue el esquema obligatorio del enunciado y separa estrictamente las responsabilidades: C se encarga de la interacción, Prolog del conocimiento y del razonamiento.

Capa	Tecnología	Responsabilidad
Usuario	—	Ingresa el vehículo, los síntomas y los resultados de pruebas; lee las conclusiones.
Front-end	C (gnu11): consola de texto y ventana Nuklear + SDL2	Menú, formularios, validación de entradas (RNF-01), presentación de resultados, mensajes de error (RNF-02) y escritura del archivo de reporte.
Integración	C: integracion/conexion.c, cJSON	Lanza swipl como proceso hijo, envía una petición JSON por línea y espera la respuesta (pipes, tiempo máximo 5 s).
Back-end	SWI-Prolog 9	Hechos, reglas, inferencia, recursividad, listas, backtracking, explicaciones, historial y texto del reporte.
Salida	—	Hipótesis con probabilidad y estado, pruebas sugeridas, acciones correctivas, explicación y reporte.

2.1 Modelo de procesos

Al iniciar, el front-end ejecuta swipl -q -g main -t halt backend/servidor_json.pl con fork y execlp, conectado mediante dos pipes. El servidor Prolog lee una línea JSON de su entrada estándar, la despacha y escribe una línea JSON de respuesta. Cada petición lleva un id que el cliente C verifica en la respuesta. Si swipl no responde en 5 segundos, se cae o devuelve una línea inválida, la interfaz muestra un diálogo de error de conexión con el botón "Reintentar", que vuelve a lanzar el proceso y repone la sesión (vehículo, síntomas y diagnóstico).

2.2 Módulos del back-end (Prolog)

Archivo	Contenido
hechos.pl	Base de conocimiento: parámetros, incompatibilidades, síntomas, componentes, conteos síntoma–componente, pruebas y acciones.
validacion.pl	validar_kb/1: revisa la consistencia de la KB (ids, rangos, conteos) en cada petición.
inferencia.pl	Capas de cálculo: verosimilitud, logit, probabilidad, relevancia de pruebas y de acciones (noisy-OR).
reglas.pl	Validación de la entrada, contradicciones (R2b), clasificación (R5), candidatos (R6/R7) y el ciclo recursivo R1–R11 con RB1/RB3.

Archivo	Contenido
consultas.pl	13 consultas lógicas sobre la KB.
explicaciones.pl	Traza narrada del razonamiento, explicación por componente, acciones, pruebas y conclusión (R10).
retroceso.pl	RB2: snapshots de la memoria de trabajo, detección de contradicciones al marcar y resolución por el usuario.
ops_sintomas.pl, memoria_x.pl	Catálogo de síntomas y memoria de trabajo X (selección actual).
pruebas_adicionales.pl	Recomendación de pruebas y registro del resultado de la prueba pendiente.
acciones.pl	Acciones correctivas de los componentes confirmados.
historial.pl	Persistencia del historial de diagnósticos (R11).
reporte.pl	Texto del reporte de un diagnóstico guardado.
vehiculo.pl, sistemas.pl	Registro del vehículo; sistema de cada componente y síntoma.
servidor_json.pl	Bucle de lectura y despacho del contrato JSON, traducción de errores.

2.3 Módulos del front-end y la integración (C)

Archivo	Contenido
frontend/main.c	Punto de entrada; verifica que exista swipl y los archivos del back-end; elige el modo consola (--consola), la ventana o el modo demostración.
frontend/consola.c/.h	Interfaz de consola: menú numerado, lectura y validación de cada entrada, resultados en texto.
frontend/menu.c/.h	Pantallas del menú y su habilitación según el estado (banderas, no reglas del dominio).
frontend/interfaz.c/.h	Dibujo de las pantallas con Nuklear sobre SDL2.
frontend/controlador.c/.h	Estado de la sesión y una función por operación del contrato; decide qué pedir y qué mostrar.
frontend/validaciones.c/.h, vehiculo.c	Validación de campos del vehículo (RNF-01).
sintomas.c, diagnostico.c, pruebas_adicionales.c, acciones.c, explicacion.c, historial.c, reporte.c	Modelos en C: validan cada respuesta de Prolog (tipos, rangos [0, 1], campos obligatorios) antes de dibujarla.
integracion/conexion.c/.h	Proceso swipl, pipes, envío y recepción de JSON, tiempo máximo.

3. Base de conocimiento

La base de conocimiento está en backend/hechos.pl como hechos de Prolog. Agregar un síntoma, componente, prueba o acción solo requiere agregar hechos: el catálogo de la interfaz y el razonamiento se actualizan sin tocar el código C.

Hecho	Significado	Cantidad
componente(Id, Nombre, Npos, Nneg)	Componente y sus conteos de casos con y sin falla (para el prior).	80
sintoma(Id, Nombre, sintoma)	Síntoma observable por el usuario.	50
sintoma(hall_P, Nombre, hallazgo)	Resultado de la prueba P (normal, leve o severo).	152
componente_sintoma(C, S, Estado, N1, N2)	Casos con el síntoma o hallazgo en ese estado, con falla de C (N1) y sin ella (N2).	652
prueba(Id, Nombre)	Prueba adicional.	152
prueba_sintoma(P, S, Aps)	Relevancia de la prueba cuando el síntoma está presente.	239
prueba_componente(P, C)	Componente que verifica la prueba.	176
accion(Id, Nombre)	Acción correctiva.	111
accion_componente(A, C, Maj)	Efectividad de la acción sobre el componente.	165
incompatibilidad(S1, S2)	Síntomas que no pueden darse a la vez.	4
parametro(Nombre, Valor)	Umbrales del motor: tau_alto 0,85, tau_bajo 0,05, tau_candidato 0,25, max_iteraciones 5, alpha 1.	5
Total		**1 786**

Ejemplo de hechos:

```
componente(liquido_frenos, 'Líquido de frenos', ...).
sintoma(pedal_freno_esponjoso, 'Pedal de freno esponjoso o blando', sintoma).
componente_sintoma(cable_embrague, hall_ajustar_tension_cable_embrague, severo, 36, 47).
prueba_componente(medir_espesor_pastillas_freno, pastillas_freno).
accion_componente(purgar_cambiar_liquido_frenos, liquido_frenos, 0.95).
incompatibilidad(no_arranca, ralenti_inestable).
```

3.1 Hechos dinámicos de la sesión

La información que cambia durante el uso se guarda como hechos dinámicos (assertz/retract), sin tocar la KB:

- vehiculo/6: el vehículo registrado (un solo registro).
- Memoria de trabajo X (memoria_x.pl): la selección de síntomas confirmada por Prolog, y sus snapshots para RB2.
- ultimo_diagnostico/3: el último diagnóstico calculado y si ya se guardó.
- diagnostico/6 y diagnostico_resultado/3: el historial, que además se escribe en datos/registros.txt.

3.2 Validación de la KB

validar_kb/1 revisa en cada petición que los ids no estén repetidos ni falten, que las relevancias y efectividades estén en [0, 1], que los conteos sean enteros no negativos y que los estados de cada hallazgo sumen los conteos del componente. Si la KB es inválida, toda operación responde kb_invalida con la lista de problemas y la interfaz lo muestra sin cerrarse.

4. Motor de inferencia y reglas

El motor combina tres capas de cálculo (definidas como reglas en inferencia.pl) con un ciclo de razonamiento recursivo (en reglas.pl). Ninguna regla del dominio está programada en C.

4.1 Capa 1: de síntomas a componentes (Naive Bayes)

Para cada componente C con evidencia se calcula su probabilidad dado lo observado, en escala logarítmica (log-odds), con suavizado de Laplace (α = 1):

```
prior(C)     = (Npos + α) / (Npos + Nneg + 2α)
P(s | C+)    = (N1 + α) / (Npos + α·|V|)      |V| = 2 para síntomas, 3 para hallazgos
P(s | C−)    = (N2 + α) / (Nneg + α·|V|)
aporte(s)    = ln( P(s|C+) / P(s|C−) )         presente o resultado de una prueba
aporte(s)    = ln( (1 − P(s|C+)) / (1 − P(s|C−)) )  ausente;   0 si es desconocido
logit(C)     = ln( prior / (1 − prior) ) + Σ aportes
P(C | S)     = 1 / (1 + e^(−logit))
```

Regla de decisión (estado_componente/2): **confirmado** si P ≥ tau_alto (0,85), **descartado** si P ≤ tau_bajo (0,05) e **incierto** en otro caso. Las probabilidades se acotan con ε = 10^{-9} para que nunca se calcule ln(0).

4.2 Capa 2: pruebas adicionales (noisy-OR)

```
Rp = 1 − Π (1 − xs · Aps)     xs = 1 si el síntoma s está presente; se excluyen las pruebas ya hechas
```

4.3 Capa 3: acciones correctivas (noisy-OR)

```
Ra = 1 − Π (1 − P(Cj|S) · Maj)     solo sobre los componentes confirmados
```

4.4 Ciclo de razonamiento R1–R11

Regla	Qué hace	Predicado
R1	Recibe la entrada y la valida (síntomas existentes, estados válidos, sin repetidos).	validar_entrada/2
R2–R5	Construye la memoria de trabajo X, calcula P(C|S) y clasifica cada componente con evidencia.	clasificar/2, probabilidad/3
R2b	Detecta síntomas incompatibles presentes a la vez y detiene el ciclo.	contradicciones/2
R6	Condición de parada: no quedan candidatos inciertos, o se llegó a max_iteraciones.	ciclo/2
R7	Elige el foco (primer candidato incierto con P ≥ tau_candidato) y recomienda su prueba de mayor Rp.	foco/4, pruebas_del_componente/4
R8	Si el usuario dio el resultado, lo agrega a X como hallazgo y vuelve a R2.	paso/3
R9	Calcula las acciones de los confirmados, ordenadas por Ra.	acciones_ordenadas/2
R10	Explica el razonamiento (aportes de cada síntoma y traza).	explicaciones.pl
R11	Guarda el diagnóstico en el historial.	guardar_diagnostico/1

4.5 Retrocesos

- **RB1, retroceso por refutación:** si la prueba deja al foco descartado ($P \leq 0{,}05$), se vuelve al punto de decisión y se pasa al siguiente candidato, conservando el hallazgo.
- **RB3, retorno tras confirmar:** si el foco queda confirmado ($P \geq 0{,}85$), se vuelve al punto de decisión para buscar otros componentes afectados.
- **RB2, retroceso por contradicción:** al marcar un síntoma incompatible con otro presente, se restaura X al snapshot anterior y el usuario elige cuál de los dos es correcto (o cancela).

4.6 Reglas compuestas

Varias reglas combinan otras reglas para deducir conclusiones que no están almacenadas como hechos:

Regla	Combina
probabilidad/3	logit/3 ← contribuciones/3 ← contribucion/4 ← verosimilitud/4 y el prior.
clasificar/2	tiene_evidencia/2, probabilidad/3 y estado_componente/2 sobre todos los componentes.
candidatos/2	clasificar/2 y el parámetro tau_candidato.
confirmados/2	clasificar/2 filtrado por estado confirmado.
relevancia_prueba/3	prueba_sintoma/3, la memoria X y complemento/2.
pruebas_del_componente/4	prueba_componente/2, relevancia_prueba/3 y las pruebas ya hechas.
relevancia_accion/3, acciones_ordenadas/2	accion_componente/3, los confirmados y complemento/2.
contradicciones/2	incompatibilidad/2 y la memoria X.

5. Recursividad

El enunciado pide resolver dos problemas con recursividad (RF-09). El sistema resuelve estos dos, cada uno con su caso base y su condición de terminación:

5.1 Problema 1: el ciclo de razonamiento R1–R11

El ciclo se llama a sí mismo después de aplicar el resultado de cada prueba (R8), con una memoria de trabajo más grande. Es recursión mutua entre ciclo/2 y paso/3:

```
ciclo(E, Final) :-
    E = e(X, _, Iter, _, _, _),
    parametro(max_iteraciones, Max),
    candidatos(X, Cands),
    (   Cands == []                  % caso base 1 (R6): no quedan inciertos
    ->  Final = final(true, sin_inciertos, E, ninguna)
    ;   Iter >= Max                  % caso base 2 (R6): máximo de iteraciones
    ->  Final = final(false, max_iteraciones, E, ninguna)
    ;   foco(E, Cands, E1, C)        % R7
    ->  paso(E1, C, Final)           % caso recursivo: paso/3 vuelve a llamar a ciclo/2
    ;   Final = final(false, sin_pruebas, E, ninguna)   % caso base 3: sin pruebas
    ).
```

Terminación: cada llamada recursiva aplica una prueba nueva (la iteración aumenta en 1 y la prueba se agrega a las ya hechas, que no se repiten), así que el ciclo termina a más tardar en max_iteraciones (5). También termina si no quedan candidatos, si no hay pruebas disponibles o si falta el resultado de la prueba recomendada (prueba_pendiente).

5.2 Problema 2: combinación de evidencias sobre listas

La probabilidad y las relevancias combinan una lista de evidencias de largo variable. Se resuelve con recursión sobre la lista [H|T]: la suma de aportes del logit y el producto del noisy-OR.

```
% contribuciones(+C, +X, -Lista): un aporte por cada dato de la memoria de trabajo.
contribuciones(_, [], []).                          % caso base: lista vacía
contribuciones(C, [S-E|X], [S-L|Ls]) :- contribucion(C, S, E, L), contribuciones(C, X, Ls).

sumar([], 0).                                       % caso base
sumar([N|Ns], T) :- sumar(Ns, T0), T is T0 + N.

% complemento(+Ts, -Pr): producto de (1 - Ti), usado por Rp y Ra (noisy-OR).
complemento([], 1).                                 % caso base: producto vacío
complemento([T|Ts], Pr) :- complemento(Ts, Pr0), Pr is Pr0 * (1 - T).
```

Terminación: cada llamada recibe la cola de la lista, que es estrictamente más corta; con la lista vacía se aplica el caso base.

5.3 Otras reglas recursivas

Además de estos dos problemas, el back-end tiene 34 predicados recursivos sobre listas. Algunos ejemplos:

- primer_candidato/5 busca el siguiente candidato válido del punto de decisión (RB1/RB3).
- pasos/5 reconstruye la memoria de trabajo prueba por prueba para narrar la traza.
- leer_terminos/2 lee el historial línea por línea, validando cada término.
- acciones_dict/3, realizadas/3 y componentes/3 arman las listas que se envían a la interfaz y al reporte.

6. Listas y backtracking

6.1 Operaciones con listas

Operación	Uso en el sistema
Recursión [H|T]	sumar/2, complemento/2, contribuciones/3, primer_candidato/5 y los 34 predicados de la sección 5.3.
findall/3	Reunir los componentes con evidencia, las pruebas de un componente, las acciones de los confirmados, los eventos de la traza.
sort/4, msort/2	Ordenar componentes por probabilidad, pruebas por Rp y acciones por Ra (de mayor a menor).
partition/4	Separar síntomas y hallazgos de la entrada en diagnosticar/2.
member/2, memberchk/2	Recorrer candidatos y buscar el estado de un síntoma en la memoria X.
append/3, length/2, exclude/3	Unir secciones del reporte, tomar los 3 mayores aportes (R10), quitar la prueba pendiente de las sugeridas.

6.2 Escenarios de backtracking

Con los síntomas del ejemplo del PDF §6 (chirrido_frenos y pedal_freno_esponjoso presentes), el punto de decisión tiene los candidatos Pastillas (63,3 %), Líquido (44,2 %) y Discos (30,6 %):

1. **RB1:** la prueba "Medir el espesor de las pastillas" da normal y Pastillas baja a 3,9 %: queda descartada, se retrocede al punto de decisión y se pasa a Líquido de frenos.
2. **RB3:** la prueba del líquido da severo y Líquido sube a 93,1 %: queda confirmado, se vuelve al punto de decisión y se evalúa Discos.
3. **RB2:** el usuario marca "El motor no arranca" y luego "Ralentí inestable": la incompatibilidad restaura X al estado anterior y el usuario elige cuál es correcto.

Además, el razonamiento aprovecha el backtracking nativo de Prolog:

- findall/3 sobre componente_sintoma/5 y prueba_componente/2 explora todas las soluciones de cada consulta (inferencia múltiple, RF-07).
- verosimilitud/4 falla cuando la KB no tiene fila para un síntoma y contribucion/4 pasa a la cláusula siguiente (el dato no aporta).
- foco/4 y primer_candidato/5 recorren la lista de candidatos hasta encontrar uno que siga siendo válido.

La traza de cada diagnóstico registra los retrocesos (eventos apilar, prueba, rb1, rb3, rb2), y la pantalla Explicación los muestra resaltados.

7. Consultas lógicas

backend/consultas.pl define 13 consultas sobre la KB, y el contrato JSON expone 19 operaciones que la interfaz ejecuta de forma dinámica (RF-06).

Consulta	Resultado
sintomas_de(C, Ss)	Síntomas asociados a un componente.
componentes_de(S, Cs)	Componentes asociados a un síntoma.
pruebas_para(C, Ps)	Pruebas que verifican un componente.
componentes_de_prueba(P, Cs)	Componentes que verifica una prueba.
sintomas_de_prueba(P, Ss)	Síntomas que vuelven relevante una prueba, con su relevancia.
acciones_para(C, As)	Acciones de un componente, de mayor a menor efectividad.
mejor_accion(C, A)	La acción más efectiva para un componente.
incompatibles_con(S, Ss)	Síntomas incompatibles con S.
prior(C, P)	Probabilidad previa del componente.
sintomas_comunes(C1, C2, Ss)	Síntomas que comparten dos componentes.
componentes_sin_pruebas(Cs)	Componentes que no tienen ninguna prueba.
componentes_mas_probables(X, N, Cs)	Los N componentes más probables para una memoria X.
hallazgos_de_prueba(P, Hs)	Estados posibles del hallazgo de una prueba.

8. Explicación del razonamiento

explicaciones.pl genera los textos desde la KB y la traza del motor; si cambia un hecho, cambia el texto. Tipos de explicación (el enunciado pide al menos 5):

Tipo	Ejemplo (caso del PDF §6)
Estado inicial (R2–R5)	Estado inicial: "Pastillas de freno" 63,3 % (incierto), "Líquido de frenos" 44,2 % (incierto) y "Discos de freno" 30,6 % (incierto).
Prueba y resultado (R7–R8)	Iteración 1: foco "Pastillas de freno" (63,3 %). Se realiza la prueba "Medir el espesor de las pastillas de freno" y el resultado es "normal" (aporte −3,8 al logit): baja de 63,3 % a 3,9 %.
Retroceso RB1	"Pastillas de freno" queda descartado (3,9 %, menor o igual que el umbral de 5,0 %). Se retrocede al punto de decisión.
Retroceso RB3	"Líquido de frenos" queda confirmado (93,1 %, mayor o igual que el umbral de 85,0 %). Se vuelve al punto de decisión.
Retroceso RB2	Al marcar "Ralentí inestable" se detectó que es incompatible con "El motor no arranca"; la memoria de trabajo se restauró.
Aportes de un confirmado (R10)	"Líquido de frenos" (93,1 %): el resultado de "Revisar el nivel y estado del líquido de frenos" aportó +2,8 y "Pedal de freno esponjoso" aportó +2,2 al logit.
Por qué una acción	"Purgar y cambiar el líquido de frenos" (relevancia 88,5 %): atiende "Líquido de frenos" (93,1 %, con efectividad 0,95).
Por qué una prueba	Se sugiere "Medir el espesor de las pastillas de freno" (relevancia 70,0 %) porque está presente "Los frenos chirrían".
Descartado o incierto	Por qué un componente se descarta o sigue incierto, con el valor de su probabilidad.
Conclusión	Diagnóstico concluyente: posible falla en "Líquido de frenos". Es una hipótesis del sistema experto, no una certeza.

9. Integración C–Prolog

9.1 Contrato JSON

Cada mensaje es una línea JSON. Petición y respuesta:

```
{"id": 2, "op": "diagnosticar", "args": {"sintomas": {"chirrido_frenos": "presente"}}}
{"id": 2, "estado": "ok", "resultados": [ { "concluyente": false, "componentes": [...], ... } ]}
{"id": 2, "estado": "error", "resultados": [], "error": {"codigo": "entrada_invalida", "mensaje": "..."}}
```

estado es ok, sin_resultados o error. El detalle de cada operación está en el README del repositorio.

9.2 Operaciones

Operación	Feature	Descripción
ping	F00	Prueba de conexión.
registrar_vehiculo, consultar_vehiculo	F04	Vehículo de la sesión.

Operación	Feature	Descripción
catalogo_sintomas, marcar_sintoma, quitar_sintoma, limpiar_sintomas, fijar_seleccion	F05	Catálogo y selección de síntomas.
resolver_contradiccion, cancelar_contradiccion	F11	Retroceso RB2.
diagnosticar	F02/F06	Ciclo R1–R11 con hipótesis alternativas.
recomendar_pruebas, registrar_resultado_prueba	F07	Pruebas adicionales y su resultado.
explicar	F08	Traza y explicación.
acciones_correctivas	F09	Acciones con Ra.
guardar_diagnostico, listar_diagnosticos, cargar_diagnostico	F10	Historial persistente.
reporte_textual	F13	Texto del reporte.

9.3 Manejo de errores (RF-13)

Situación	Respuesta del sistema
Entrada inválida (síntoma inexistente, estado fuera de rango, campo del vehículo inválido)	C la rechaza antes de llamar a Prolog; si llega a Prolog, responde entrada_invalida con la lista de problemas y el estado no cambia.
JSON mal formado u operación desconocida	json_invalido u op_desconocida; el servidor sigue atendiendo.
Sin hipótesis	sin_resultados; la interfaz sugiere agregar síntomas.
KB inválida	kb_invalida con los problemas; el diagnóstico queda bloqueado con un mensaje.
swipl caído o sin respuesta en 5 s	Diálogo de error de conexión con "Reintentar"; la sesión se repone al reconectar.
Historial con líneas dañadas	registros_corrupto: se muestran los registros válidos y se avisa.
Error al escribir un archivo	error_escritura o el motivo (sin permiso, disco lleno); no queda un archivo a medias.
Errores del dominio	sin_diagnostico, sin_vehiculo, ya_guardado, prueba_no_pendiente, ya_registrada, sin_guardar, no_encontrado.

10. Front-end en C

El front-end ofrece dos interfaces escritas en C sobre el mismo controlador (controlador.c): cada operación se envía a Prolog por el contrato JSON y la interfaz solo lee, valida y muestra. No hay clases ni orientación a objetos: estructuras y funciones. Las condiciones en C solo controlan el menú, las validaciones y el flujo; las conclusiones siempre vienen de Prolog.

10.1 Interfaz de consola

Cumple la restricción 42 (interfaz funcional mediante consola). Se inicia con `make consola` o `./sistema_experto --consola` y no abre ninguna ventana. Muestra el estado de la sesión y un menú numerado:

```
--- Menú principal ------------------------------------------------
  Prolog: conectado · Vehículo: sin registrar
  Síntomas: 0 · Diagnóstico: no calculado

  1. Registrar o editar el vehículo
  2. Seleccionar síntomas
  3. Diagnosticar
  4. Pruebas adicionales  (requiere diagnóstico)
  5. Explicación del razonamiento  (requiere diagnóstico)
  6. Acciones correctivas  (requiere diagnóstico)
  7. Guardar diagnóstico
  8. Historial de diagnósticos
  9. Reporte  (requiere diagnóstico)
  0. Salir
Opción [0-9]:
```

Cada opción tiene su submenú (por ejemplo, Síntomas permite marcar por número o por identificador, quitar, limpiar y ver el catálogo por sistema). Las entradas se validan antes de consultar a Prolog: una opción fuera de rango o un síntoma que no existe se vuelven a pedir. Una contradicción (RB2) se resuelve con un menú, y si `swipl` deja de responder se ofrece reintentar. El fin de la entrada estándar cierra el programa sin error, lo que permite probarlo con un guion de entradas (`pruebas/pruebas_integracion.c`, caso I11).

10.2 Ventana gráfica

La ventana usa Nuklear (biblioteca de interfaz de un solo encabezado) sobre SDL2 y ofrece las mismas operaciones en 8 pantallas:

Pantalla	Función	Habilitada cuando
Vehículo	Registrar y editar las características del vehículo.	Siempre
Síntomas	Catálogo por sistema; presente, ausente o desconocido; diálogo RB2.	Siempre
Diagnóstico	Hipótesis ordenadas por probabilidad; guardar en el historial.	Hay síntomas
Pruebas	Prueba recomendada, registrar su resultado, pruebas realizadas.	Hay diagnóstico
Explicación	Traza por iteración y resumen por componente confirmado.	Hay diagnóstico
Acciones	Acciones correctivas por Ra y su origen.	Hay diagnóstico
Reporte	Vista previa y exportación del reporte.	Hay diagnóstico
Historial	Diagnósticos guardados y su detalle.	Siempre

Validaciones en C antes de consultar a Prolog (RF-03): campos del vehículo (texto, año entre 1950 y el año siguiente, kilometraje, combustible), síntomas que existan en el catálogo, estados válidos, al menos un síntoma presente o ausente, que el resultado registrado sea de la prueba pendiente, y que toda probabilidad o relevancia recibida esté en [0, 1].

11. Persistencia y reportes

Historial (RF-14): al guardar, Prolog agrega al final de datos/registros.txt un hecho diagnostico(Id, Fecha, vehiculo(...), Concluyente, Motivo, Iteraciones) y un diagnostico_resultado(Id, Clave, Valor) por cada parte del resultado. El archivo se lee al iniciar el servidor, así que los diagnósticos se conservan entre ejecuciones. Cada línea se valida al leerla; las dañadas se omiten y se informan sin perder las válidas. Si el archivo o la carpeta no existen, se crean.

Reporte (RF-15): reporte_textual arma el texto desde el registro guardado (fecha, vehículo, síntomas, pruebas realizadas, componentes con probabilidad y estado, acciones con Ra, traza resumida y estado final). C lo muestra y lo escribe en datos/reportes/diagnostico_AAAAMMDD_HHMMSS.txt, sin sobrescribir nunca un archivo existente.

Las variables de entorno SE_REGISTROS y SE_REPORTES cambian esas rutas; las pruebas automáticas las usan para no tocar los datos reales.

12. Pruebas

make test ejecuta todas las pruebas automáticas. Resultado de la última ejecución:

Conjunto	Cantidad	Resultado
Prolog (plunit), pruebas/pruebas_prolog.pl	307 pruebas	0 fallos
Controlador C, pruebas/pruebas_controlador.c	89 comprobaciones	0 fallos
Validadores C, pruebas/pruebas_c.c	70 verificaciones	0 fallos
Integración C ↔ swipl real, pruebas/pruebas_integracion.c	143 verificaciones	0 fallos

Los 15 casos de prueba documentados, con entrada, expectativa, resultado y evidencia, están en el documento "Casos de prueba" y en pruebas/casos_prueba.txt.

13. Cumplimiento de requisitos

13.1 Requisitos mínimos del motor lógico (sección 5)

Elemento	Mínimo	Sistema	Dónde
Hechos de conocimiento	40	1 786	hechos.pl
Reglas lógicas	20	421 cláusulas de reglas	15 módulos del back-end
Predicados propios	15	216 + 13 consultas	backend/
Consultas distintas	10	13 consultas y 19 operaciones	consultas.pl, contrato JSON
Reglas recursivas	2	2 problemas (sección 5) y 34 predicados recursivos	reglas.pl, inferencia.pl
Operaciones con listas	3	6 tipos (sección 6.1)	Todo el back-end
Escenarios de backtracking	3	RB1, RB2, RB3 y backtracking nativo	reglas.pl, retroceso.pl

Elemento	Mínimo	Sistema	Dónde
Reglas de inferencia compuestas	5	9 (sección 4.6)	inferencia.pl, reglas.pl
Casos de prueba	15	15 documentados y 609 verificaciones automáticas	pruebas/
Explicaciones de conclusiones	5	10 tipos (sección 8)	explicaciones.pl

13.2 Requisitos funcionales

Código	Requisito	Implementación
RF-01	Menú principal	Menú numerado de la consola (consola.c) y panel lateral de la ventana con 8 pantallas (menu.c, interfaz.c).
RF-02	Gestión de información	Vehículo, síntomas y resultados de pruebas.
RF-03	Validación	validaciones.c y los modelos C, antes de consultar a Prolog.
RF-04	Base de conocimientos	hechos.pl (sección 3).
RF-05	Reglas de inferencia	inferencia.pl, reglas.pl (sección 4).
RF-06	Consultas lógicas	19 operaciones JSON y 13 consultas (sección 7).
RF-07	Inferencia múltiple	diagnosticar devuelve todas las hipótesis con su probabilidad.
RF-08	Backtracking	RB1, RB2, RB3 (sección 6.2).
RF-09	Recursividad	Dos problemas recursivos (sección 5).
RF-10	Listas	Sección 6.1.
RF-11	Explicación	explicar y pantalla Explicación (sección 8).
RF-12	Recomendaciones	Pruebas adicionales (Rp) y acciones correctivas (Ra).
RF-13	Errores	Sección 9.3.
RF-14	Persistencia	datos/registros.txt (sección 11).
RF-15	Reportes	datos/reportes/ (sección 11).

13.3 Restricciones técnicas (sección 7)

N.º	Restricción	Cumplimiento
38	Front-end exclusivamente en C	Todo el front-end está en C (gnu11).
39	Back-end exclusivamente en Prolog	Todo el back-end está en SWI-Prolog.
40	Sin otros lenguajes	Solo C y Prolog; Nuklear y cJSON son bibliotecas en C.
41	Sin clases ni orientación a objetos	Estructuras y funciones en C; predicados en Prolog.

N.º	Restricción	Cumplimiento
42	Interfaz funcional mediante consola	Interfaz de consola en C: make consola o ./sistema_experto --consola (menú de texto numerado, sección 10.1). La ventana gráfica en C ofrece además las mismas funciones.
43	Razonamiento en Prolog	Todo el razonamiento está en backend/.
44	Sin reglas del dominio en C	C solo valida, controla el flujo y dibuja lo que responde Prolog.
45	Recursividad en Prolog	Sección 5.
46	Demostrar backtracking	Sección 6.2; visible en la traza.
47	Sin respuestas precargadas	Toda conclusión se calcula desde los hechos en cada petición.
48	Sin bibliotecas externas de IA	El cálculo está escrito en Prolog con aritmética básica.
49	SWI-Prolog	SWI-Prolog 9.0.

14. Decisiones de diseño

1. **Naive Bayes en log-odds con Laplace.** Permite combinar muchos síntomas de forma explicable: cada síntoma suma un aporte al logit, y ese aporte es justamente la explicación. El suavizado evita probabilidades 0 o 1 cuando faltan casos.
2. **Umbral tau_candidato (0,25).** Solo se investigan los inciertos con evidencia y $P \geq 0{,}25$; sin él, varios componentes cercanos al 15 % alargarían el ciclo sin aportar.
3. **Comunicación por pipes con JSON en lugar de la interfaz C de SWI-Prolog.** Mantiene los dos lenguajes en procesos separados, facilita las pruebas automáticas de cada lado y permite recuperarse si swipl se cae.
4. **El estado vive en Prolog.** La selección de síntomas, el vehículo y el último diagnóstico se guardan en Prolog; C solo refleja lo que Prolog confirma y, si swipl se reinicia, reenvía la sesión.
5. **RB2 al marcar cada síntoma.** La contradicción se detecta cuando ocurre (ahí existe "la última entrada" que se debe deshacer) y el diálogo pide al usuario elegir.
6. **Solo se registra la prueba pendiente.** El motor aplica un hallazgo cuando R7 pide esa prueba; registrar otra no cambiaría el diagnóstico.
7. **Sistema de cada componente.** La KB no tiene sistemas; sistemas.pl los asigna con una tabla y deduce el sistema de cada síntoma.
8. **Reporte desde el registro guardado.** Así el reporte coincide con el historial y es el mismo al reabrir la aplicación.
9. **Dos interfaces sobre el mismo controlador.** La consola cumple la restricción 42 al pie de la letra y la ventana facilita la demostración; ambas llaman a las mismas funciones del controlador, así que el comportamiento y las validaciones son idénticos.
10. **Calibración de umbrales.** make calibrar muestra la ventana válida de tau_alto y tau_bajo; los valores 0,85 y 0,05 clasifican bien los casos de prueba.

15. Estructura del repositorio

```
Sistema-Experto-Automotriz/
├── frontend/       main.c, consola.c/.h, menu.c/.h, interfaz.c/.h, validaciones.c/.h,
│                   controlador.c/.h y modelos de cada pantalla
├── backend/        hechos.pl, reglas.pl, inferencia.pl, consultas.pl,
│                   explicaciones.pl y demás módulos
├── integracion/    conexion.c, conexion.h (cJSON en vendor/)
├── datos/          registros.txt y reportes/ (se crean al usar el programa)
├── pruebas/        pruebas_prolog.pl, casos_prueba.txt, pruebas en C, evidencia/
├── documentacion/  manual técnico, manual de usuario, casos de prueba
├── Makefile
└── README.md
```

Para compilar y ejecutar, ver el manual de usuario: make build, make consola, make run y make test.